

MLP-Offload: 面向大模型预训练的多层级、多路径卸载，突破 GPU 内存墙

Avinash Maurya

Argonne National Laboratory
Lemont, IL, USA
amaurya@anl.gov

Franck Cappello

Argonne National Laboratory
Lemont, IL, USA
cappello@mcs.anl.gov

M. Mustafa Rafique

Rochester Institute of Technology
Rochester, NY, USA
mrafique@cs.rit.edu

Bogdan Nicolae

Argonne National Laboratory
Lemont, IL, USA
bogdan.nicolae@acm.org

摘要

由于大模型的大小增长速度快于 GPU 内存的增长速度，训练参数量超过多个 GPU 聚合内存的大模型变得日益必要。为此，最先进的方法提出了多级主机内存或磁盘卸载技术。尽管采用了先进的异步多级读/写策略，此类卸载策略仍会在训练的关键路径中引入显著的 I/O 开销，从而导致迭代速度变慢。为此，我们提出 *MLP-Offload*——一种新颖的多级、多路径卸载引擎，专为在资源受限环境下优化大模型训练而设计，以缓解 I/O 瓶颈。我们得出若干关键观察结果，用以指导 *MLP-Offload* 的设计，例如：更新阶段的 I/O 开销主导了整个迭代时间；第三级远程存储层的 I/O 带宽仍未被充分利用；并发卸载所引发的资源争用进一步加剧了 I/O 瓶颈。受这些洞见驱动，我们设计并实现了 *MLP-Offload*，以缓存高效且并发可控的方式将优化器状态卸载至多个存储层级，从而缓解反向传播与更新阶段的 I/O 瓶颈。在高达 280B 参数规模的模型上进行的评估表明，相较于当前最先进的大模型训练运行时，*MLP-Offload* 可实现 $2.5\times$ 倍的更快迭代速度。

CCS Concepts

• 信息系统 → 数据管理系统; 存储管理; • 计算方法 → 分布式人工智能.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SC '25, St Louis, MO, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1466-5/2025/11

<https://doi.org/10.1145/3712285.3759864>

Keywords

并行与分布式大模型及 Transformer 训练，多级异步卸载，混合精度训练，混合内存与存储层级管理，低资源成本高效训练

ACM Reference Format:

Avinash Maurya, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2025. *MLP-Offload*: 面向大模型预训练的多层级、多路径卸载，突破 GPU 内存墙. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3712285.3759864>

1 引言

大模型 (LLMs) 凭借文本摘要和知识蒸馏技术，彻底改变了多个领域的研究范式，使研究人员能够更高效地处理复杂的科学文献。一种自然的演化趋势是朝着更通用的基础模型 (FMs) 发展，这些模型能够捕捉不同数据模态之间的复杂相关性（例如通过交叉注意力机制），从而开始对科学进步产生更大的影响。

在追求提升涌现行为（即并非显式训练但因模型规模巨大以及在训练过程中接触海量数据而自发产生的能力）的过程中，函数模型 (FMs) 的规模和复杂度持续增加，需要更大的训练基础设施并产生巨大的成本。没有大规模高性能计算 (HPC) 系统，预训练甚至微调都变得不可行：ChatGPT-3 的训练使用了超过 10,000 块 GPU [12]，每次训练成本超过 400 万美元。Meta 的 LLaMA 模型（2023 年 2 月发布）也遵循相同模式：它使用 2,048 块 Nvidia A100 GPU 在 1.4 万亿 token 上进行训练，耗时约 21 天 [41]。更糟糕的是，大模型 (LLMs) 在参数量和训练数据大小方面都在迅速增长。例如，据估计 ChatGPT-4 的规模比 ChatGPT-3 大 $10\times$ ，且训练所用数据量比 ChatGPT-3 多 $570\times$ 。在

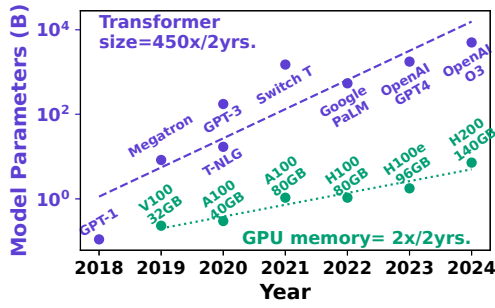


图 1: 模型与 GPU 内存增长。

这些情况下，训练或微调函数模型所需的规模 and 成本变得极为昂贵，难以承受。

动机: GPU 内存墙. 大量前沿技术已被提出, 以高效利用大规模 GPU 所提供的计算能力与巨大并行性。例如, DeepSeek [23] 在此方向引入了多种技术。然而, 随着 GPU 计算效率的提升, 瓶颈逐渐转向 GPU 内存容量不足的问题。例如, Gopher-280B 模型 [28] 需要 4.8 TB [30] 的内存, 这表明训练数百亿参数级别的模型需要达到太字节量级的 GPU 内存。这一瓶颈导致所谓的“内存墙”现象: 模型参数及激活值、优化器状态等辅助数据结构 ($450 \times / 2$ 年) 的增长速度远超新一代设备的 GPU 内存增长速度 ($2 \times / 2$ 年) [10, 25, 38]。如图 1 所示, 这一趋势是不可持续的。此外, 大多数高性能计算数据中心需要同时为多个用户运行不同工作负载提供服务 [6]。因此, 相较于长时间排队等待获取大量 GPU 资源, 能够在有限数量的 GPU 上训练和微调大模型 (LLMs) 和函数模型 (FMs) 的能力显得尤为可取 [9, 15]。

为应对这一挑战, 先进的大语言模型训练框架 (例如 DeepSpeed [31] 和 Megatron [33]) 引入了多种技术, 如 ZeRO 冗余优化 (例如将大型优化器状态在数据并行副本间划分) [29]、量化、混合精度训练 [33] 等。然而, 这些方法仍不足以缓解对大量 GPU 内存的需求。因此, 通常还需将重要的数据结构 (模型参数、优化器状态) 卸载至较慢的主机内存 (DRAM) [26] 以补充资源。当内存约束更为严苛时, 聚合的 GPU 和主机内存容量仍显不足, 促使进一步将数据结构卸载至第三级存储层级 (例如 NVMe 设备) [21, 30, 32]。

尽管较慢的内存和存储层级具有显著更高的容量, 但其输入/输出 (I/O) 带宽也显著更低。例如, 在单个节点的聚合 GPU 内存中运行 200 亿参数的 LLaMA 模型预训练而无任何卸载时, 平均每次迭代耗时为 0.4 秒。若将更新阶段使用的优化器状态卸载至主机内存, 则平均迭代时间增加至 3.7 秒。若进一步卸载至节点本地的 NVMe, 则平均迭代时间上升至 67 秒 (§ 3.1)。这种严重的 I/O 瓶颈导致整体训练速度相比无卸载情况下降约 170 $\times$ 。在更大规模下, 例如跨多个节点训练 700

亿参数的 LLaMA 模型时, 通信开销开始主导迭代时间, 从而分摊了 NVMe 卸载的相对成本。结果, 与仅使用 GPU 训练相比, 采用 NVMe 卸载的训练速度下降幅度减少了约 7 $\times$, 凸显了分布式环境下通信与 I/O 瓶颈之间的相互作用。

因此, 即便在大规模场景下, 如何减少使用第三级存储时的 I/O 瓶颈, 仍是亟待解决的重要挑战。只有克服这一问题, 才能在不带来显著性能惩罚的前提下, 充分受益于显著更高的总体内存容量。

当前最先进方法的局限性. 现有的先进大模型训练运行时, 如微软的 DeepSpeed [31]、PyTorch 的完全分片数据并行 (FSDP) [53]、Colossal-AI [19] 等, 引入了针对模型和优化器状态的新型异步卸载技术。尽管 FSDP 仅支持将数据卸载到主机内存, 但 DeepSpeed 和 Colossal-AI 运行时支持 NVMe 卸载。具体而言, 模型参数和优化器状态被划分为模型分片, 这些分片在工作进程 (每个 GPU 一个) 之间进行分配, 并通过模型并行协同工作。相应地, 每个模型分片被进一步分解为更小的块, 称为“子组”, 这些子组结合混合精度技术逐一处理, 流程如下: 在前向传播和反向传播阶段, 使用 GPU 上的 FP16 参数副本计算每个子组的梯度; 随着反向传播的推进, 梯度以 FP32 格式被刷新至主机内存, 再从主机内存写入三级存储。随后, 在 CPU 上执行更新阶段, 以避免主机内存与 GPU 内存之间产生过量的数据传输, 从而抵消 GPU 的计算加速优势。在更新阶段, 每个子组的 FP32 模型参数、优化器状态以及梯度从三级存储加载至主机内存, 由 CPU 完成更新操作, 之后将更新后的模型参数和优化器状态刷新回三级存储, 同时将对应的 FP16 版本推送到 GPU。通过使用高效的流水线和异步 I/O 技术, 借助 DeepNVMe [32] 等专用引擎, 使计算与 GPU 内存、主机内存及三级存储之间的数据传输重叠。然而, 这仍不足以缓解三级存储有限的 I/O 带宽问题。此外, 计算节点配备多个共享同一主机内存和三级存储的 GPU, 导致 I/O 带宽竞争问题未得到缓解。另一个现有先进技术的局限性在于, 由于对梯度处理策略不够优化, 以及未能有效复用已存在于主机内存中的子组, 造成了主机内存与三级存储之间不必要的来回移动。

核心见解与贡献. 在本工作中, 我们提出了一种名为 MLP-Offload 的新方法, 旨在缓解当前最先进的卸载技术所面临的上述局限性。这些技术利用三级存储来提升整体记忆容量和 I/O 带宽。具体而言, 我们基于一个关键观察: 高性能计算 (HPC) 系统的外部存储 (如并行文件系统 (PFS) 和对象存储) 在大型语言模型 (LLM) 训练期间通常处于低利用率状态 (除偶尔的检查点保存外 [49]), 因此可以与节点本地的 NVMe 存储相结合, 显著提升 I/O 带宽。结合并发控制以缓解对 I/O 带宽的竞争, 并通过更好地复用从一

次迭代到下一次迭代的缓存子组, 这些思路使得 *MLP-Offload* 相比现有最佳方法实现了 $2.5\times$ 的加速。

我们总结了主要贡献如下:

- (1) 我们在训练的不同阶段对资源利用情况进行深入分析, 此时优化器状态被卸载到 SSD 上。特别地, 我们提出了几个基本观察结果: 用于异步数据传输缓存缓冲的 CPU 内存, 在每次迭代中都会出现抖动; 梯度的放大和卸载是冗余的; 现代并行文件系统 (PFS) 能够提供数 GB/s 级别的并行带宽 (例如使用 *Infini-band*), 但这些带宽未被利用; 尽管传输与基于 CPU 的优化器更新重叠, 由于磁盘写入速度较慢, CPU 核心仍处于空闲状态 (§ 3.1)。
- (2) 基于上述表征, 我们提出了一系列设计原则: 使用虚拟层级的统一多级、多路径异步卸载; 针对多路径 I/O 优化的虚拟层级并发控制; 面向缓存的模型子组处理顺序; 以及更新过程中的延迟就地混合精度梯度转换 (§ 3.2)。这些设计原则得到了 § 3.3 中详述的 I/O 性能模型的补充。
- (3) 我们提出 *MLP-Offload*, 这是一个开源实现, 遵循了将现有最先进大语言模型训练运行时 (如 *DeepSpeed* 和 *Megatron*) 集成的设计原则。我们的实现坚持低层级优化, 例如在 *libaio* 中采用进程独占的多线程共享锁机制——一个经过优化的异步 POSIX I/O 库; 以及高效地非顺序排列异步预取和写回磁盘的操作 (§ 3.5)。
- (4) 我们在一系列实验中评估了 *MLP-Offload*, 在 $32\times$ A100-40GB GPU 上对最多包含 280B 参数的模型进行预训练。与当前最先进的 *DeepSpeed* (§ 4) 相比, 我们的方法分别将反向传播和参数更新阶段加速了 $13.5\times$ 和 $2.3\times$, 并将端到端训练速度提升了 $2.5\times$ 。

2 背景与相关工作

数据、流水线与张量并行. 用于数据、流水线和张量的各种并行技术已被广泛采用, 以加速大型模型 (如卷积神经网络 (CNNs) [52]、深度学习推荐模型 (DLRMs) [50]、大语言模型 (LLMs) [51] 以及大视觉模型 (LVMs) [53]) 的训练。数据并行通过运行多个模型副本, 每个副本在不同的输入小批量上并行执行前向和反向传播, 并在迭代结束时的更新阶段进行同步, 以整合所有小批量所学成的模式, 从而加速训练。流水线 [13] 和张量 [5, 33] 并行技术将大型模型跨有限内存容量的 GPU 进行划分。虽然流水线并行通过将模型层的子集垂直分配到特定 GPU 上来实现模型划分, 而张量并行则通过在所有可用 GPU 之间水平划分模型来实现。数据、流水线和张量并行的结合, 通常称为“三维并行”, 被用于在数千个 GPU 上有效扩展大规模训练, 适用于 CNN、LVM、LLMs、DLRMs 等。然而, 鉴于大模型规模持续增长及其在各领域的快速应用, 为在资源受限环境下运行, 需要超越三维并行的优化。

ZeRO 冗余消除. 当前最先进的大语言模型训练运行时, 例如 *PyTorch* 的完全分片数据并行 (FSDP) [53]、*Microsoft DeepSpeed* 的零冗余 (ZeRO) [29]、*Colossal-AI* [19], 采用冗余消除技术移除跨数据并行秩之间的模型和优化器状态的冗余部分, 以最小化 GPU 内存消耗。为此, 广泛使用的 [34, 45] 大语言模型训练运行时 *DeepSpeed* 提出了三种消除数据并行秩间冗余的阶段: ZeRO-1 仅拆分优化器状态; ZeRO-2 拆分优化器状态和梯度; ZeRO-3 则拆分优化器状态、梯度和模型参数 [29]。如图 2(a) 所示, 在 N 个 GPU 上对 P 参数的模型进行全冗余消除 (ZeRO-3) 训练, 可实现 $\sim [\mathcal{O}(P)/N]$ 量级的内存节省, 但代价是通信开销增加了 $1.5\times$ [29]。

ZeRO-3 技术将模型状态划分到多个 GPU 上, 需要频繁执行散播-汇集操作以按需重构层, 显著增加了通信开销。因此, 尽管 ZeRO-3 具有内存效率优势, 却无法与依赖高效阶段间通信的流水线并行无缝结合。相反, ZeRO-3 采用混合模型和数据并行策略, 其中在 N 个 GPU 上进行训练通常会产生 N 个“虚拟”数据并行副本。与传统数据并行中模型完全复制不同, 这些副本为虚拟的, 因为模型状态会根据需要动态地在 GPU 之间获取和同步。

混合精度训练. 混合精度训练由百度和英伟达研究提出 [27], 是另一种广泛采用的提升大模型训练吞吐量并减少内存占用的方法。如图 2(b) 中左侧虚线框所示, 具体而言, 混合精度使用两份不同精度的模型参数副本: 一份为半精度 (FP16 或 BF16), 用于执行前向和反向传播; 另一份为主副本, 采用高精度 (FP32), 由优化器在更新阶段使用以保持更高的稳定性 [18]。前向和反向传播产生的活性值和梯度均采用半精度格式, 从而实现更快的通信和计算速度。低精度的 FP16 梯度会被放大至 FP32, 并由优化器用于执行更新步骤。多个实际的大模型, 如 BLOOM-176B [45]、OPT-175B [49]、GPT-3 [4] 以及 GLM-130B [47], 均采用混合精度进行预训练, 充分证明了该方法在准确率和效率方面的优势。

将模型和优化器状态划分到子组中. 为了减少计算过程中所需的中间内存, *DeepSpeed* 的 ZeRO-3 将每个秩/GPU 的模型参数和优化器状态划分为子组, 如图 2(b) 所示。这种子组分片技术是 *DeepSpeed* 运行时独有的, 在其他运行时 (如 FSDP 或 *Colossal-AI*) 中不可用。这些子组的大小 M 由用户定义, 表示每个子组中的参数总数。

子组在各 GPU 之间均匀分布。在使用混合精度进行训练时, 对于每个包含 M 个参数的子组, 前向和反向传播操作在 FP16 参数和 FP16 梯度上进行, 而更新步骤则在对应的 M 个 FP32 参数和 FP32 优化器状态上进行。这种子组式分片方式能够在最小化内存占用

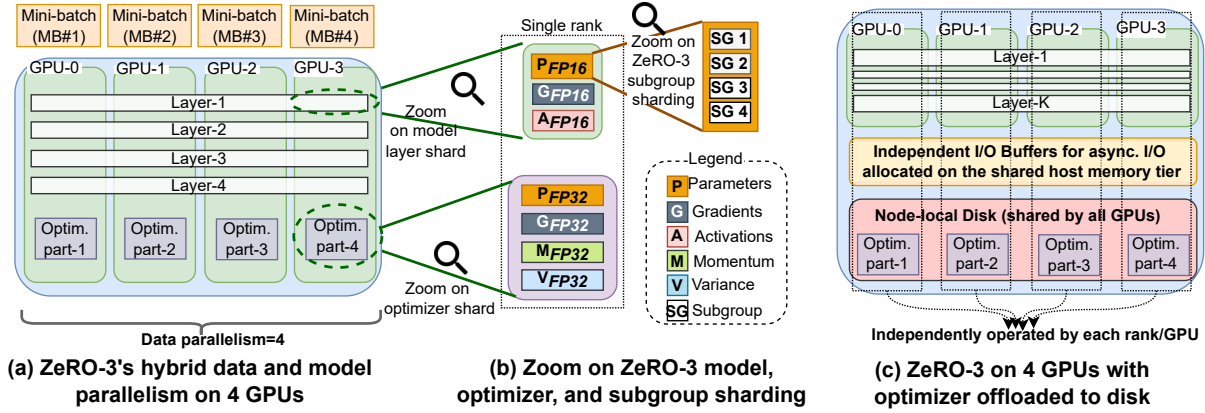


图 2: DeepSpeed ZeRO-3 显示 (a) 混合模型和数据并行；其中 (b) 模型参数、活性值、梯度和优化器状态被划分为子组；以及 (c) 优化器状态被卸载到结点本地的 NVMe。

的同时，实现高效的分段计算和通信重叠，如 ZeRO-Infinity [30] 所述。

优化器状态卸载. 优化器状态以 FP32 格式保存，其大小远大于模型参数，且仅在更新步骤中需要。因此，在内存受限的场景下，将其卸载到主机内存（以及更进一步的多级存储层次，如节点本地 NVMe）是一种实际可行的选择。子组的使用进一步促进了主机内存与其他多级存储层次之间的交换。如图 2(c) 所示。当启用优化器卸载时，更新通常在 CPU 上执行，原因在于：(a) GPU 内存通常已被 FP16 模型参数完全占用，(b) 即使按子组逐个传输 FP32 优化器状态和梯度，从 GPU 到 CPU 的传输仍会产生较高的 I/O 开销。为了高效管理这些子组更新，每个 GPU 工作进程在共享主机内存上分配一个独立的 I/O 缓冲区，其容量足以容纳可配置数量的子组。这些 I/O 缓冲区主要承担两个功能：(1) 在反向传播期间缓存 FP16 梯度，将其放大至 FP32，并异步地将 FP32 梯度写入磁盘，以便在更新步骤中使用；(2) 通过异步的子组交换机制，实现主机内存与磁盘之间的优化器子组卸载。

加速模型与优化器卸载. 近期多项研究致力于缓解由于训练、微调和推理场景中卸载操作所引发的 I/O 瓶颈问题。例如，Fuyou [21] 和 LoHan [22] 通过在 GPU-CPU-SSD 之间流水线式重叠活性值、参数和优化器状态的交换，以加速极端受限场景下的微调过程，但在这些场景中，模型或优化器状态无法完全容纳于 GPU 内存中；因此不适用于我们工作所针对的场景。Smart-Infinity [14] 利用计算存储设备（连接至 FPGA 的 NVMe）来减轻 NVMe 与 CPU 之间的 I/O 开销，从而加速更新阶段，但需要专用的 NVMe 设备。为缓解昂贵的更新阶段带来的性能下降，此前已在 ZeRO-Offload [32] 中提出异步更新技术，如单步延迟更新：它将下一次迭代在 GPU 上的执行（使用过时参数）与仍在进行中的

基于 CPU 的更新重叠。然而，由于模型状态不一致的问题，该优化后来被从运行中移除 [36]。DeepSpeed TwinFlow [43] 和 Deep Optimizer States [26] 是两项互补性工作，它们仅专注于优化纯 CPU 卸载，未涉及基于磁盘的卸载优化。

3 我们的贡献：MLP-Offload

3.1 差距分析

冗余消除与多级卸载的结合使得在通常无法容纳于聚合 GPU 内存的大规模语言模型的预训练和微调成为可能。然而，GPU 内存、主机内存以及三级存储之间的 I/O 带宽差异较大，尤其是在考虑节点本地 SSD 或其他形式的磁盘存储时尤为明显 [32]。因此，I/O 开销可能会显著延迟更新阶段，该阶段需要获取、更新并重新写入卸载的子组。

训练迭代主要由更新阶段主导. 作为第一步，我们研究了训练迭代中前向传播、反向传播和更新阶段所花费的时间分布，以检查更新阶段是否占整体迭代时长的显著比例，从而验证加速更新阶段能够加快整体训练迭代的假设。为此，我们研究了 DeepSpeed [31]，其配置为使用冗余消除以及包含本地 SSD 的多级卸载。我们在单个节点上运行实验，该节点配备 4× 张 H100-80GB GPU（见表 1），并记录了迭代的时间分布。我们确实确认，迭代主要由更新阶段主导。例如，在 40B 模型的情况下（设置和方法分别在 § 4.1 中描述），前向传播和反向传播分别耗时 0.6 秒 (0.02%) 和 28 秒 (11%)，而总迭代时间为 242 秒，更新阶段耗时 213 秒 (89%)。因此，加速更新阶段是实现更快端到端运行时间的重要步骤。

更新阶段在使用 SSD 卸载时主要由 I/O 驱动. 我们评估了四种模型配置，即 20B、40B、70B 和 120B 参

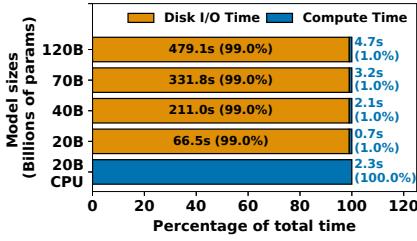


图 3: 更新阶段用于磁盘输入/输出的时间比例

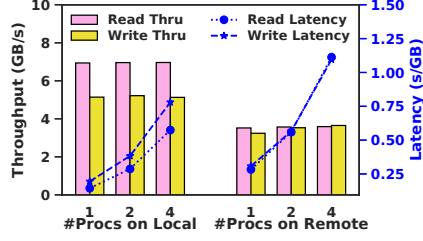


图 4: SSD (本地) 与并行文件系统 (远程) 的 I/O 带宽对比

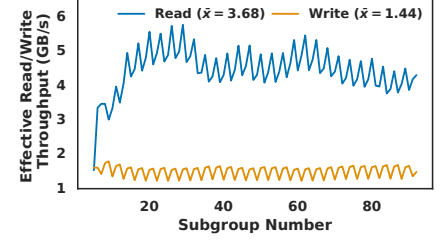


图 5: 每个子组在并发情况下的 I/O 带宽 (使用本地 SSD)

数模型 (表 2)。对于 20B 模型 [44], 完整的 FP32 参数和优化器状态可容纳在 512 GB 主机内存中; 因此, 我们将其用作基准。更大的模型尺寸会将子组卸载到 SSD。如图 3 所示, 在 20B 模型的情况下, 更新阶段比将模型子组卸载到 SSD 的更大模型快 $\sim 30\times$ 。具体而言, 当子组被卸载到 SSD 时, 99% 的更新阶段耗时都用于 SSD I/O, 尽管 DeepSpeed 的优化 DeepNVMe 引擎能够将 I/O 与 CPU 计算重叠。因此, 我们可以得出结论: 加速从主机内存中获取和写入卸载子组的速度, 将显著加快更新阶段, 而根据之前的实验, 这将显著加速整体迭代。

不同三级存储的 I/O 带宽. 高性能计算 (HPC) 系统不仅配备了结点本地的 SSD, 还提供了外部存储, 例如并行文件系统 (PFS) 和对象存储 (如 DAOS)。尽管外部存储由所有计算结点共享, 因此会面临 I/O 竞争问题, 但在中等规模下, 它能够提供更显著的聚合 I/O 带宽提升, 以弥补结点本地 SSD 有限的 I/O 带宽。

为了量化这一机会, 我们使用微基准测试测量了结点本地 NVMe 存储与远程 PFS 的原始读写吞吐量。在某些情况下, 远程存储的吞吐量甚至高于结点本地 NVMe (例如表 1 中的 Testbed-2)。此外, 由于每个 GPU 进程独立为其优化器状态子组发起获取和刷新操作, 理解在竞争条件下的 I/O 性能至关重要。图 4 显示, 随着并发进程数量增加, 整体读写吞吐量保持不变, 但每个进程的延迟 (以线条和次要纵坐标轴表示) 因存储子系统内部的竞争而恶化。这表明共享 NVMe 带宽饱和是一个限制因素, 使得 I/O 成为大规模情况下的主要瓶颈。

指令传输带宽在卸载过程中的并发性. 最后, 我们分析了在将 40B 模型的优化器状态卸载到节点本地 NVMe 时, 训练运行时感知到的有效读写吞吐量。图 5 显示了由于运行时内存限制导致的吞吐量波动, 这使得主机内存中同时活跃的子组数量被限制为三个: 一个预取, 一个正在更新, 一个正在回写到磁盘。缓慢的回写速率导致下一个子组在前一个子组完全写入之前就被预取, 从而引起读吞吐量的间歇性峰值。然而, 聚

合后的读写吞吐量仍然受限于 NVMe 的写带宽, 进一步证实了先前实验中观察到的 I/O 瓶颈。

3.2 设计原则

基于虚拟层级的统一多级、多路径异步卸载. 利用本地 SSD 进行卸载是可扩展的, 因为可以使用模型并行独立地利用 SSD 以实现高聚合 I/O 带宽。然而, 将模型子组卸载到本地存储 (如 SSD) 会引入较高的 I/O 开销, 显著减慢更新阶段的速度, 如 § 3.1 中所述。另一方面, 外部存储具有显著潜力来弥补节点本地 SSD 有限的 I/O 带宽, 并且在预训练期间通常未被充分利用 (除了偶尔的检查点外) [46]。正是基于这一观察, 我们将其应用于扩展现有的多级卸载技术, 以支持每一级的多路径 I/O。具体而言, 我们将所有替代存储 (本地 SSD、并行文件系统 (PFS)、对象存储) 统一为一个虚拟的第三级层级, 可用于将模型子组从主机内存中卸载。然后, 基于一个 I/O 性能模型 (详细内容见 § 3.3), 我们根据各子组的 I/O 带宽成比例地将其分配到替代存储层级。通过这种方法, 我们可以并行化 I/O 操作, 使得较慢的层级与较快的层级大致同时完成, 因为它们存储的子组较少。这种 I/O 负载均衡使我们的方法避免任何层级成为瓶颈, 从而最大化 I/O 操作的加速效果, 使其与计算重叠。尽管我们以第三级层级为例说明该原理, 但其可泛化至任意层级 (例如第二级层级, 用于组合 GPU-CPU 或 GPU-GPU HBM、DDR、CXL 等)。

针对多路径 I/O 的优化虚拟层并发控制. 为了实现模型并行的高可扩展性, 我们允许每个工作进程 (通常关联到单个 GPU) 独立地对备用存储执行 I/O 负载均衡。通过这种方法, 我们避免了昂贵的全局同步需求。然而, 每个计算结点包含多个 GPU, 这意味着多个进程将竞争共享的备用存储 I/O 带宽。因此, 我们在结点级别引入一种轻量级并发控制策略, 确保每个计算结点上每次只有一个工作进程可以访问特定的备用存储。通过这种对每个备用存储的独占访问限制, 工作进程能够获得完整的 I/O 带宽, 从而实现更低的延迟 (图 4) 和 I/O 负载均衡。同时, 其他工作进程可以自

由地使用所有可用的 CPU 核心来计算预取组的更新，并并行使用其他备用存储，从而自然地实现全局 I/O 负载均衡。需要注意的是，一个工作进程不限于使用单一 I/O 线程访问备用存储；它可以利用备用存储的首选 I/O 并行性（例如，使用多个 I/O 线程时 PFS 性能更优 [11]）。此外，如果首选 I/O 并行性无法被单个工作进程饱和，则可以将备用存储进一步划分为多个虚拟层级。

缓存友好的模型子组处理顺序. 自适应优化器（例如 ADAM [17]）更新阶段的一个重要观察是，计算具有令人尴尬的并行性：每个模型子组都有其对应的参数、优化器状态和梯度，可以独立应用以获得新的模型参数和优化器状态。因此，各个子组被独立处理的顺序无关紧要，不会影响准确率或收敛性。我们利用这一观察结果，设计了一种缓存友好的更新策略，尽可能地利用位于快速层级（主机内存）上的子组。具体而言，在第一次迭代中，我们按子组 ID 从小到大的顺序开始处理。更新阶段结束后，高 ID 的子组将驻留在快速（缓存）层级，而低 ID 的子组则因被逐出至慢速层级以腾出主机内存空间给更高 ID 的子组而位于慢速层级。在第二次迭代中，与现有最先进的方法不同，我们反转了子组的处理顺序。这使得更新阶段显著加速，因为大量高 ID 的子组已存在于快速层级，从而减少了由于内存抖动导致的 I/O 开销。第二次迭代后，低 ID 的子组最终位于快速层级，这意味着我们可以在第三次迭代中再次利用递增顺序。因此，我们持续交替采用子组 ID 的升序和降序处理，以最大化缓存带来的优势。

更新阶段的延迟就地混合精度梯度转换. 如 §2 所述，卸载通常与混合精度训练结合实现。在此情况下，业界领先的方法通常将反向传播过程中计算得到的 FP16 梯度从 GPU 推送到主机内存，并在其中转换为 FP32。随后，这些数据被刷新至第三级虚拟层级。

然而，在更新阶段，每个被卸载到第三级虚拟层级的子组都需要重新加载回主机内存。由于该子组由 FP32 优化器状态（参数、动量和方差）组成，且因包含 FP32 梯度而变得更大，因此每次获取操作比仅持有优化器状态时更慢。

另一方面，现代 CPU 上进行的 FP16 到 FP32 梯度转换具有高通量（表 1 中 Testbed-1 的测试结果为 65 GB/s），其速度比获取吞吐量高出一个数量级。因此，我们提出一种替代策略：在反向传播期间，我们将 FP16 梯度直接存储在主机内存中——这本来就需要预留足够的空间来容纳所有子组的 FP16 梯度，以支持梯度累积。

在更新阶段，与现有方法不同，我们仅获取子组（不含 FP32 梯度），并在线将 FP16 梯度转换为其 FP32 形式（使用与 DeepSpeed [31, 43] 相同的标准化数值原

语）。因此，我们同时加速了反向传播过程（因为我们消除了可能延迟反向传播的大规模异步 FP32 梯度刷新操作，这些操作若未能完全与计算重叠，就会造成延迟），以及更新阶段（因为就地进行的 FP16 到 FP32 梯度转换开销通常远小于从低速层级获取 FP32 梯度的 I/O 开销）。

3.3 子组分配性能模型

为了实现能够利用多个可选存储选项的虚拟层级的负载均衡，通过聚合多路径 I/O，我们提出一种方法，将模型子组按每个可选存储的 I/O 带宽成比例分配。具体而言，我们假设每个工作节点将其模型分片划分为 M 个大小相等的子组，这通常是实现计算负载均衡的情况。此外，我们假设一个由 N 个存储层级 P_i 组成的虚拟层级，其中 $0 \leq i < N$ ，各自具有相应的 I/O 带宽 B_i ——即读取或写入吞吐量的极小点。那么，分配给每个存储层级 P_i 的子组数量 T_i 可表示为：

$$T_i = \left\lfloor \frac{M \cdot B_i}{\sum_{i=0}^N B_i} \right\rfloor, \text{ adjusted such that } \sum_{i=0}^N T_i = M \quad (1)$$

公式 1 背后的直观思想是，将大致等于其 I/O 带宽对总聚合 I/O 带宽贡献的子组数量分配给每个备选存储。这使得来自不同备选存储的子组并行获取和刷新操作大致在同一时间完成。因此，这将降低由于某个备选存储拖慢进度而其他存储仍处于空闲状态所导致的计算停顿的似然。

最初，使用微基准测试对每种备选存储的 B_i 进行测试。随后，在第一次迭代之后，根据子组刷新和获取的平均观测 I/O 带宽对 B_i 进行调整。这确保了我们的方法能够适应可能影响某些备选存储选项的任何 I/O 带宽趋势变化。例如，一个在批量作业预约期间独占的本地 SSD 不会发生 I/O 带宽变化。然而，一个并行文件系统可能会受到来自不同用户拥有的多个批量作业的 I/O 压力，在这种情况下，更新后的 B_i 可以调整 T_i 的值，根据各子组的 I/O 带宽将它们重新划分到不同的虚拟层级中。

除了加快反向传播和更新阶段外，MLP-Offload 中的虚拟存储层级还通过将部分优化器状态预加载到持久化存储中，加速了检查点过程。这一机制可被多层次异步检查点引擎（如 DataStates-LLM [24]）利用，在不可变的前向和反向传播期间，将模型和优化器状态的剩余部分从 GPU 内存、主机内存以及非持久化存储层级（如本地磁盘）中刷新出去。

3.4 MLP-Offload 的系统组成

我们通过图 6 展示如何结合上述设计原则，对比 DeepSpeed 的 ZeRO-3 与我们的方法在端到端迭代执行过

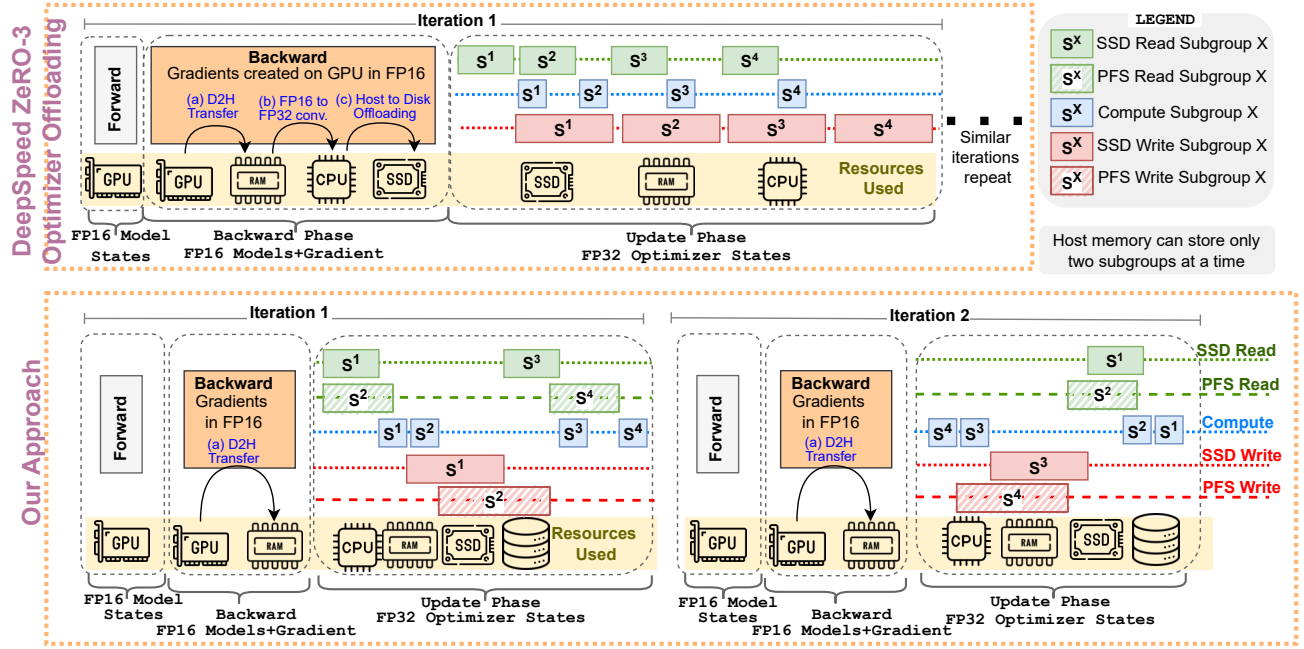


图 6: 示例说明: *MLP-Offload* 提出的设计原则与当前最先进方法 (DeepSpeed ZeRO-3) 的应用对比。

程中的差异。具体而言, 我们假设一种简化的大型语言模型 (LLM) 架构, 其中每个 GPU 上的模型分片由四个子组构成, 分别表示为 $S^1 \dots S^4$ 。当使用模型并行时, 其他进程也会并行发生相同的模式。DeepSpeed ZeRO-3 在反向传播期间为每个子组计算 FP16 梯度, 然后在后台将其刷新至主机内存。在主机端, FP16 梯度被转换为 FP32 并刷新至 NVMe。在更新阶段, 每个子组 (包含 FP32 参数、动量、方差和梯度) 通过流水线异步地从 NVMe 加载到主机内存: 一旦 S^1 出现在主机内存中, 相应的并行多核 CPU 计算即被触发以更新模型和优化器状态。与此同时, S^2 被加载至主机内存。当计算完成后, 子组的新参数被加载到 GPU, 而优化器状态则在丢弃 FP32 梯度后刷新至 SSD。在整个更新阶段, 数据加载、更新和刷新操作相互重叠。在下一个迭代中, 该模式序列重复进行。

在我们提出的方法中, 我们创建了一个虚拟的第三级存储层级, 该层级将本地 NVMe 与并行文件系统 (PFS) 相结合。初始时, 子组在主机内存中创建, 并根据 § 3.3 中讨论的性能模型, 被刷新到 NVMe 或 PFS。FP16 模型参数被复制到 GPU。随后, 训练迭代可以开始。与 ZeRO-3 不同, 反向传播不会将梯度上采样并刷新到存储层级。

Algorithm 1 提供了高层次的伪代码描述, 补充了图 6。具体而言, 在更新阶段, S^1 和 S^2 通过分离的 I/O 路径从 NVMe 和 PFS 并行获取。一旦 S^1 完成 (第 #5 行), 对应主机累加缓冲区中的梯度即从 FP16 上采样至 FP32 (第 #6 行), 随后开始更新计算 (第 #7

Algorithm 1: Optimizer Multi-Tier Offloading and Updates

Input: $iter$: training iteration number; $\langle S \rangle$: Optimizer subgroups; $\langle B \rangle$: I/O bandwidth of local and remote tiers. $f2h$, $h2f$, and $h2d$, denote file-to-host, host-to-file (for NVMe and PFS), and host-to-device (GPU) transfers, respectively.

```

1 Function run_update( $iter$ ,  $\langle S \rangle$ ,  $\langle B \rangle$ ):
2    $update\_order \leftarrow \{0, 1, \dots, |S| - 1\}$  or
    $\{|S| - 1, |S| - 2, \dots, 0\}$ 
   // reversed at each update phase to maximize host
   memory hits
3   for  $i \in update\_order$  do
4      $pt \leftarrow storage\_tier(i)$ 
5      $s \leftarrow f2h\_prefetch\_wait\_subgrp(i)$ 
6      $s.grad \leftarrow host\_grad\_accum_{FP16}[i].to(FP32)$ 
7      $cpu\_update\_kernel(s)$ 
8      $async\_h2d\_transfer(s.params.to(FP16))$ 
9      $t \leftarrow assign\_storage\_tier(i, B)$ 
10     $async\_h2f\_flush(s, t)$ 
11     $async\_f2h\_prefetch(next\_subgroup(i), pt)$ 

```

行)。接着, 在第 #8 行启动异步的主机到设备的下采样 FP16 模型参数传输, 之后在第 #9 行使用公式 1 确定 S^1 的新存储层级 t 。随后, 在第 #10-11 行启动异步地将 S^1 刷新至其新层级 t , 以及异步预取前一层级

S^1 的下一组 pt 。如图 6 所示, 由于 S^2 的获取在 S^1 的更新阶段之前完成, 因此 S^2 的更新计算可在 S^1 的更新计算结束后立即开始。同时, S^1 的刷新和 S^4 的预取可独立进行, 再次通过分离的 I/O 路径。尽管异步预取和刷新操作可以入队, 但当与其他节点本地进程竞争时, 可能因层级独占并发控制机制 (§ 3.2) 而被延迟。类似地, S^3 和 S^4 也有机会利用分离的 I/O 路径进行获取。然而, 与 ZeRO-3 不同的是, 我们无需将 S^3 和 S^4 刷新到虚拟层级, 因为可以在下一个更新阶段 (第 #2 行) 切换顺序, 直接从主机内存使用 S^4 和 S^3 。

3.5 实施与集成

我们在 *MLP-Offload* 中实现了所提出的设计原则。¹ 作为一个独立的开源库, 与 DeepSpeed ZeRO-3 集成。具体而言, 该集成扩展了 DeepSpeed 的 DeepNVMe 无主卸载引擎, 该引擎利用 libaio [8]——一个由内核加速的异步 I/O 库——支持跨进程 (每个进程映射到 GPU) 和存储层级的多路径并行读写操作。除了受益于内核加速的 I/O 外, DeepNVMe 的基于 C++ 的实现避免了 Python 全局解释器锁 (GIL) 以及 PyTorch 的池化内存管理所带来的低效问题。在此基础上, *MLP-Offload* 通过显式的基于池的分配机制, 实现了高效的主机缓冲区管理, 支持异步获取/刷新操作, 从而在进程共享存储层级时实现细粒度并发, 并降低 I/O 竞争。

MLP-Offload 可通过 DeepSpeed 运行时配置中的两个 JSON 键值对启用和配置。在 ZeRO-3 初始化期间, 我们为每个进程实例化多个卸载引擎对象, 对应于存储层级的数量。每个卸载对象被分配一个专用的主机缓冲区, 以支持异步预取和延迟刷新。主机缓冲区大小可根据可用主机内存、每个节点的进程数以及存储层级数量进行配置。此外, *MLP-Offload* 允许用户指定子组在存储层级间的分布, 该分布由性能模型 (§ 3.3) 指导。例如, `/local/` 和 `/remote/` 目录之间 2:1 的划分确保每有一个子组远程存储, 就有两个子组卸载到本地磁盘。除了管理多个卸载引擎外, *MLP-Offload* 还基于卸载比例动态预测子组的预取顺序以及何时延迟刷新已更新的子组。尽管专为 Megatron-DeepSpeed 与 ZeRO-3 设计, *MLP-Offload* 的核心原理使其可扩展至其他训练运行时, 例如在 Colossal-AI [19] 中的 TensorNVMe [37], 只需通过指定多个 `DiskOffloader` 对象创建虚拟的第三级存储层级, 从而将由我们的性能模型决定的相应子组卸载至其上。

表 1: 测试平台配置

Feature ↓ Testbed →	Testbed-1 [2]	Testbed-2 [3]
GPUs	4× H100-80GB	4× A100-40GB
Pinned D↔H B/W (GB/s)	55	25
Number of CPUs cores	96	32
Per node host memory (GB)	512	512
NVMe Read Write thput (GB/s)	6.9 5.3	13.5 4.8
Parallel File System (PFS)	VAST FS	Lustre FS
PFS Read Write throughput (GB/s)	3.6 3.6	6.9 13.7

4 性能评估

4.1 方法学

实验设置. 我们在 Testbed-1 (ANL JLSE) [2] 和 Testbed-2 (ALCF Polaris) [3] 上进行了实验, 每个结点分别配备 4× H100-80GB 和 4× A100-40GB GPU, 具体配置如表 1 所示。Testbed-1 与 Testbed-2 平台的主机内存与聚合 GPU 内存之比分别为 1.6:1 (类似于 AWS p4de.24xlarge) 和 3.2:1 (类似于 AWS p5.48xlarge), 均不足以容纳 ZeRO-Infinity [30] 中描述的 8:1 优化器状态与模型状态内存比例, 因此必须进行 NVMe 离线存储。两个平台均配备 2× 块 RAID 挂载的 1.6 TB NVMe M2 SSD 作为本地存储, 其读写吞吐量列于表 1。Testbed-1 结点采用 2× 颗 Intel(R) Xeon(R) Platinum 8468 处理器, 共 96 个 CPU 核心; 而 Testbed-2 结点采用 1× 颗 AMD EPYC 7543P 处理器, 共 32 个 CPU 核心。我们使用 Testbed-1 进行小规模实验, 使用 Testbed-2 进行可扩展性实验。在外部存储作为替代离线存储层级方面, Testbed-1 配备容量为 1 PB 的 VAST [42] 并行文件系统, 由 4 个 DNode 组成, 其读写吞吐量大致相当于 AWS 的高级 FSX Lustre 文件系统 [1] 的速度。Testbed-2 则由 100 PB 存储容量的 HPE ClusterStor E1000 平台构成, 通过 160 个 OST (对象存储目标) 实现, 其读写吞吐量见表 1。

对比方法. 我们将 *MLP-Offload* (如图 6 (底部) 所示) 与 *DeepSpeed ZeRO-3* 进行对比, 后者是当前大语言模型训练最先进方法的一个典型实现, 如图 6 (顶部) 所示。具体而言, DeepSpeed ZeRO-3 支持将优化器状态卸载到 NVMe, 这一点与 Colossal-AI [19] 相似。我们采用其异步卸载引擎, 即 DeepNVMe, 通过重叠所有三个操作 (获取、更新、刷新) 来加速更新阶段, 代价是需要额外的主机内存用于异步数据传输。鉴于在持有部分子组时主机内存已耗尽, 我们将这一先进的异步引擎视为代表性基准。

模型与数据集. 我们在评估中使用的模型配置, 基于真实世界的大型语言模型训练场景, 总结在表 2 中。我们不考虑小于 40B 的模型, 因为它们的优化器状态足够小, 可以放入主机内存 (512 GB)。我们使用 OSCAR-en 数据集的一个子集, 包含 79K 条记录, 该数据集包

¹<https://github.com/DataStates/artifacts/blob/main/MLP-Offload>

表 2: 用于评估的模型。 N_L : 层数; D_H : 隐藏维度; AH : 注意力头数。

Model	40B[44]	52B[20]	70B[40]	100B[44]	120B[35]	130B[48]	280B[28]
N_L	128	64	80	124	96	70	72
D_H	5120	8192	8192	8192	10240	12288	16384
AH	40	64	64	64	80	96	128

含在 Bloom 模型的代码库中 [45], 使用 LLaMA2 [39] 的默认分词器将数据集预处理为 token。除非另有说明, 与 OPT 训练配置 [49] 类似, 我们将默认序列长度设为 2048, 微批量大小设为 1, 以避免任何配置下出现内存溢出 (OOM) 错误。

运行时配置. 如第 2 节所述, DeepSpeed 不支持流水线并行与 ZeRO-3 的组合使用, 而 ZeRO-3 负责模型和优化器状态的分片。因此, 在单结点实验中, 我们采用数据并行与 ZeRO-3 的组合方式, 将模型参数、梯度和优化器状态分片以适配 GPU 内存。在弱可扩展性实验中, 我们采用张量并行 (结点内) 与数据并行 (结点间) 相结合的方法, 以最大化性能和内存节省 [30, 33]。

在每次实验中, 所选结点上的所有 GPU 均被充分利用, 且每个 GPU 唯一地关联到一个进程。对于所有模型, 均启用了本地 NVMe 卸载, 基于异步方法实现子组的获取、刷新和更新操作之间的重叠。我们的方法额外访问了 PFS。为了促进子组的预取和延迟刷新以实现异步卸载, 预先分配了可配置数量的固定主机缓冲区, 从而使主机内存尽可能被充分利用 ($> 90\%$ 内存利用率), 适用于所有对比方法。由于每个模型在运行时级别保留不同数量的缓冲区 (如梯度累积、全部归约等), 因此异步卸载的主机缓冲区大小在不同模型间有所不同。有关运行时所消耗的 GPU/主机内存的详细分解, 请参见 ZeRO-Offload [32]、ZeRO-Infinity [30] 以及 DeepSpeed 内存估计量 [7]。

在我们的评估过程中, 我们确保聚合的 GPU 内存足以存储以下内容: (1) FP16 模型参数; (2) 前向传播生成的活性值检查点; 以及 (3) 至少一个子组在反向传播过程中生成的 FP16 梯度, 该梯度会异步刷新至位于主机内存中的梯度累积缓冲区。我们还确保主机内存足够大, 以容纳运行时级别的缓冲区 (例如, 梯度累积、全部归约桶等 [7]), 并至少保留三个子组以支持异步更新: 前一个子组被延迟刷新到磁盘, 当前子组正在更新, 下一个子组则从本地 NVMe 预取。

尽管子组大小不会影响迭代时长、收敛或准确率, 如 [14, 26] 所述, 但较小的子组能够实现更优的卸载子组的 I/O 与计算重叠。因此, 对于所有方法, 我们采用 1 亿可训练参数的子组大小, 而非 DeepSpeed 默认的子组 10 亿参数, 这有助于我们方法的更好负载均

衡。类似于 Turing-NLG 17.2B、GPT-3 175B、BLOOM-176B [45], 我们使用了活性值检查点技术, 在牺牲反向传播阶段 33% 额外重计算的前提下降低 GPU 内存占用 [30]。这是在 GPU 内存资源有限情况下进行训练的常见选择。然而, 我们的方法与活性值检查点技术是互补的, 即使不使用活性值检查点, 也能产生类似的效果。

关键性能指标. 我们使用以下指标来评估上述方法: (1) 单次训练迭代的平均运行时间 (按前向传播、反向传播和参数更新阶段分别分解); (2) 参数更新吞吐量 (以每秒更新的百万参数数表示); (3) 在读取和写入磁盘卸载子组时观察到的有效读/写吞吐量; (4) 优化器状态在不同层级间的分布情况。这些指标对于理解采用我们方法进行大语言模型训练的端到端性能与可扩展性, 以及突出影响最终结果的重要中间步骤至关重要。

4.2 结果: 模型大小可扩展性

我们首先测量迭代时间, 按前向、反向和更新阶段的持续时间, 针对测试床-1 (表 2) 上单个 $4 \times H100$ 结点的模型大小变化进行分析 (表 1)。每次实验运行 10 次迭代, 其中前 2 次为预热迭代, 报告结果取剩余 8 次的平均值。我们将模型大小在 40B 到 120B 之间变化, 以确保单个子组的所有 FP16 参数和梯度均能容纳在聚合的 320 GB GPU 内存中。随着模型大小增加, 卸载强度也随之上升, 从而在卸载压力增大时凸显出所比较方法的有效性 (在 120B 参数规模下, 优化器状态达到 1.8 TB)。

如图 7 所示, 随着模型大小的增加, 迭代持续时间呈上升趋势 (52B 与 40B、120B 与 100B 之间存在轻微例外, 因为它们 Transformer 层较少但隐藏维度更多)。正如预期, 由于卸载机制的存在, 更新阶段持续时间最长, 而前向传播几乎可以忽略不计。对于 DeepSpeed ZeRO-3, 反向传播开始变得明显, 而我们的方法将其降至可忽略水平。此外, 我们观察到, 我们的方法使更新阶段加速了高达 $2.4\times$, 从而使整体迭代速度比 DeepSpeed ZeRO-3 快达 $2.7\times$ 。

为了进一步解释这些结果, 我们在图 8 中展示了更新吞吐量。作为参考, 当模型参数和优化器状态完全存在于 GPU 和主机内存中时, GPU 上的吞吐量为 $\sim 40000M$ 参数/秒, 而 CPU 上约为 $\sim 8000M$ 参数/秒。在启用卸载后, 即使在 CPU 上, 更新吞吐量也下降了一个数量级, 这证实了瓶颈并非来自计算本身, 而是由于 NVMe 和/或 PFS 的慢速 I/O 所致。此外, 对于所有模型规模, 更新吞吐量保持相对稳定 (每个子组的更新吞吐量不变, 仅不同模型的子组数量发生变化)。总体而言, MLP-Offload 的更新吞吐量比 DeepSpeed ZeRO-3 高出 $\times 1.8\times - 2.4\times$ 。

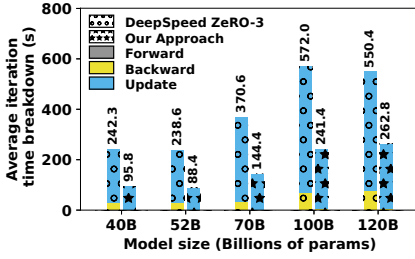


图 7: 模型大小缩放时的平均迭代时间分解。

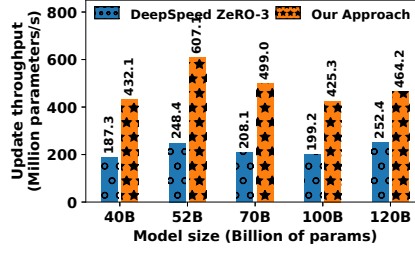


图 8: 缩放模型大小时观察到的平均更新吞吐量。

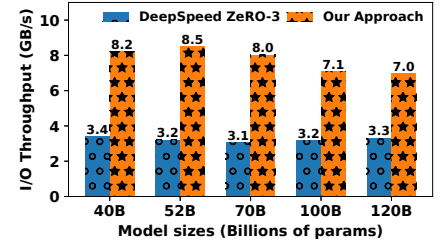


图 9: 不同模型大小的有效输入/输出吞吐量。

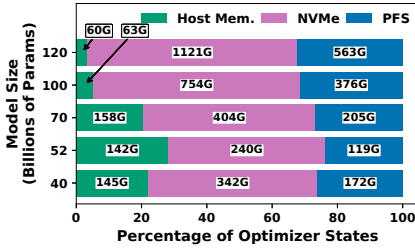


图 10: 不同层级之间优化器状态的分布。

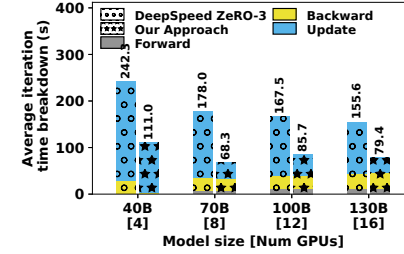


图 11: 弱缩放: 随着模型规模增大, 不同 GPU 数量下的迭代时间。

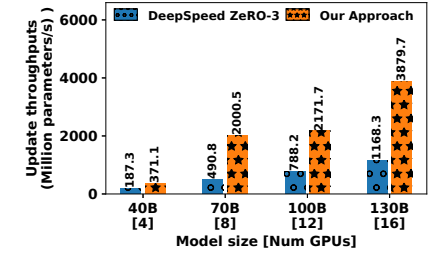


图 12: 弱缩放: 随着模型大小增加, 更新每秒的吞吐量 (# GPUs)。

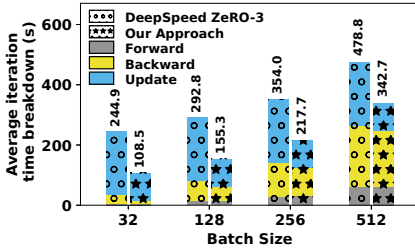


图 13: 40B 模型在不同批量大小下的平均迭代时间。

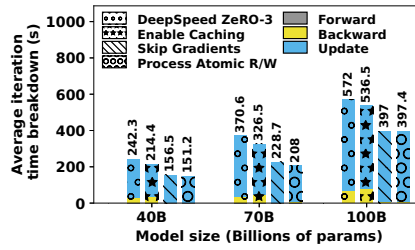


图 14: 结点本地 NVMe 性能消融分析。

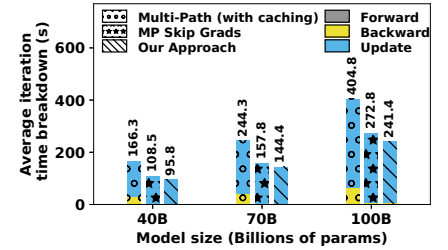


图 15: 结点本地 NVMe 与 PFS 的性能消融分析。

4.3 结果: 输入/输出和存储层级负载

如前所述, 更新吞吐量受到 I/O 瓶颈的影响。因此, 接下来我们讨论 *MLP-Offload* 与 DeepSpeed ZeRO-3 在持续 I/O 吞吐量方面的表现。与图 4 中讨论的微基准测试不同, 端到端训练过程中, I/O (异步预取和刷新) 与计算重叠进行, 这引入了额外开销。具体而言, I/O 吞吐量按 $2 \times \text{subgroup_size_bytes} / (\text{read_time} + \text{write_time})$ 计算, 并对所有子组取平均值。由于每个子组都需要被读取和写入, 因此大小翻倍。图 9 展示了随着模型大小增加, 所有子组的聚合 I/O 吞吐量。有趣的是, DeepSpeed ZeRO-3 方法表现出的 I/O 吞吐量为 ~ 3.2 GB/s, 远低于 Testbed-1 (表 1) 上 NVMe 的峰值写入速度 (5.3 GB/s)。这是因为来自所有进程的并行多线程读写在 CPU-NVMe 互联 (本例中为 PCIe) 以及 NVMe 存储子系统上造成

了竞争。相反, *MLP-Offload* 通过使用 PFS 缓解了本地 NVMe 的压力, 从而同时受益于多层次卸载和各层级更优的利用率, 最终快了 $2.6 \times$ 倍。随着模型大小的增加, *MLP-Offload* 的有效 I/O 吞吐量略有下降, 因为优化器状态较小比例可缓存在主机内存中, 从而降低了缓存的有效性。然而, *MLP-Offload* 对于较大模型仍比 DeepSpeed ZeRO-3 方法实现了 $\sim 2 \times$ 倍的 I/O 加速。

为了说明多级卸载的影响, 我们重点展示了优化器状态在不同存储层级间的分布情况。主机内存的大部分被 DeepSpeed 运行时用于设置 ZeRO-3 特定的数据结构, 这些结构消耗了 250–350 GB 的主机内存, 其大小与模型规模成比例, 如 DeepSpeed 的内存估计量 [7] 所报告。剩余的主机内存则用于缓存子组和异步 I/O 操作。图 10 展示了在每次迭代中, 优化器状态在主机内存、本地 NVMe 和 PFS 之间的分布百分比。与 *MLP-Offload* 不同, DeepSpeed ZeRO-3 因子组按顺序更新而

造成主机缓冲区的缓存抖动, 并未使用 PFS, 导致所有优化器状态在每次迭代中都必须读取和写回本地 NVMe。本地 NVMe 与 PFS 之间优化器状态的分配比例验证了我们性能模型的有效性 (§ 3.3), 表明其 2:1 的 NVMe 到 PFS 卸载比例与表 1 中的读写吞吐量一致。

4.4 结果: 弱可扩展性

接下来, 我们通过将模型大小与结点数量成比例地变化, 研究 *MLP-Offload* 的弱可扩展性。如 § 2 中所述, DeepSpeed 未实现流水线并行与模型参数及优化器状态分片的结合。因此, 我们在同一计算结点上的四个共置 GPU 之间使用张量并行, 在计算结点之间使用数据并行。

ZeRO-3 在数据并行秩之间进行参数分片, 需要频繁执行散射集合操作, 导致前向和反向传播阶段的通信开销更高, 以换取内存节省。因此, 这项可扩展性研究对于理解在大规模情况下, 更高的通信开销是否抵消了 *MLP-Offload* 在反向传播和参数更新阶段所获得的收益至关重要。

在本实验中, 我们在测试平台 2 (表 1) 上扩展至 8 个结点 ($32 \times \text{A100-40GB GPU}$)。从表 2 中选择能够在聚合 GPU 内存中以 FP16 格式容纳的最大模型, 确保模型大小与结点数量成比例增加: 1 结点对应 40B, 2 结点对应 70B, 3 结点对应 100B, 4 结点对应 130B, 8 结点对应 280B。图 11 显示, 随着 GPU 数量增加, 迭代时间减少。在此情况下, 前向和反向传播中的通信开销相较于由于高性能计算基础设施中通常配备的快速互连 (如 Slingshot、InfiniBand) 导致的卸载优化器状态更新的 I/O 瓶颈而言并不显著。此外, 结点数量的增加使得能够独立访问本地 NVMe 进行 I/O 操作, 从而加速了子组更新。因此, 即使在大规模下, *MLP-Offload* 的迭代速度也比 DeepSpeed ZeRO-3 快达 $2\times$ 。为解释这一趋势, 我们在图 12 中分析了更新吞吐量。正如预期, 更新吞吐量随资源 (I/O 带宽和 CPU) 增加而提升; 结合迭代时长来看, 这证实了 I/O 仍是主要瓶颈。

除了展示 *MLP-Offload* 在模型规模增大时相对于 ZeRO-3 的可扩展性优势外, 弱可扩展性实验还突显了与纯 GPU 训练场景相比, *MLP-Offload* 采用 NVMe 卸载的成本效益。例如, 在不进行卸载的情况下训练 70B 模型需要 ~ 80 块 A100-40GB GPU 的聚合内存 [24], 单次迭代耗时 24 秒。相反, 如图 11 所示, 通过基于 NVMe 的优化器卸载, ZeRO-3 可以使用少 $10\times$ 块 GPU 运行, 但每次迭代需 168 秒, 即慢 $7\times$ 倍。相比之下, *MLP-Offload* 仅慢 $4.8\times$ 倍, 从而在少用 $10\times$ 块 GPU 的情况下实现 $5\times$ 倍的加速——与纯 GPU 训练相比, 成本效益提升了 $2\times$ 倍。

4.5 结果: 梯度累积的可扩展性

我们接下来研究梯度累积的影响, 这是一种流行的技术 [45], 通过在每次更新阶段之前执行多次前向和反向传播来减少更新阶段的数量 (从而降低卸载的影响)。虽然迭代次数并未减少, 但其效果相当于使用更大的小批量进行训练 (当 GPU 内存不足时, 这是不可行的另一种方法)。目标是证明, 尽管更新阶段的频率降低, 我们的方法仍能相较于最先进水平带来显著的端到端加速。

我们在 Testbed-1 ($4 \times \text{H100 GPU}$) 上对 40B 模型进行实验, 该配置可容纳批量大小为 8 个样本的最小批量, 超过此值时会出现 GPU 内存不足的情况。因此, 当在 4 个 GPU 上以数据并行方式运行时, 随着前向和反向传播次数 (1–16) 的增加, 等效批量大小在 32–512 范围内增长。如图 13 所示, 即使使用梯度累积来分摊昂贵更新阶段的成本, *MLP-Offload* 仍比 DeepSpeed ZeRO-3 至少快 40% 。

4.6 结果: 消融研究

在上一组实验中, 我们进行了消融研究, 以理解每项优化对我们的设计原则 (§ 3) 所提出方法的影响。我们考虑了三种模型, 即 40B、70B 和 100B, 分别代表小规模、中规模和大规模模型, 在 Testbed-1 上进行测试。图 14 展示了当优化器状态仅卸载到节点本地 NVMe 时, 各项优化的累积影响 (即逐步激活)。标记为 *Enable Caching*、*Skip Gradients* 和 *Process Atomic R/W* 的方法分别对应于 § 3.2 中讨论的设计原则: 面向缓存的子组重排、延迟的就地混合精度梯度转换以及优化的虚拟层级并发控制。如图所示, 逐项激活每一项优化进一步缩短了迭代时间, 这意味着每项优化均独立贡献于加速, 即使没有并行文件系统 (PFS), 也能实现最高达 $1.6\times$ 的加速比, 优于 DeepSpeed。图 15 展示了相同优化累积效果, 但此时启用了 PFS (多路径模式)。在此情况下, 激活全部三项优化等价于我们的方法。与图 14 相比, 多路径并行 I/O 进一步将迭代速度提升了 $1.6\times$, 最终使迭代速度相比 DeepSpeed ZeRO-3 提升了 $2.5\times$ 。

5 结论

在本文中, 我们提出了一种新颖的技术——*MLP-Offload*, 并将其作为库实现, 可与最先进的大模型运行时集成, 以支持可扩展的训练和微调。具体而言, 我们旨在将优化器状态卸载至多层次、多路径的内存与存储层次结构中, 从而在 GPU 内存受限的情况下加速大规模模型的训练。在此背景下, 现有方法在将优化器状态卸载至存储层级时面临显著的 I/O 瓶颈, 原因在于全精度优化器状态的大小 (比 FP16 参数大 $8\times$ 倍) 远超主机内存容量 (通常仅比聚合的 GPU 内存大 $2\times$ 倍),

因此必须卸载至三级存储层级（如节点本地 NVMe 设备），而这些层级的 I/O 带宽低几个数量级。为缓解这些 I/O 瓶颈，*MLP-Offload* 提出了若干设计原则，包括多层次多路径异步卸载、多路径 I/O 的并发控制、面向缓存的子组更新重排序，以及动态就地混合精度梯度转换。这些设计原则被实现为 DeepSpeed 卸载引擎的模块化扩展。在 40B–280B 参数模型上的大量评估表明，与 DeepSpeed ZeRO-3 相比，不同配置下训练迭代速度提升 2.5×。受此结果鼓舞，我们下一步计划探索下一代计算快速链接（CXL）内存池中的并行 I/O 路径，并将 *MLP-Offload* 与其它卸载运行时、框架及加速器进行集成。特别值得关注的是，在 I/O 竞争环境下对全局共享替代存储层级行为的深入研究，例如并行文件系统和对象存储。在此情况下，我们计划探索如何缓解可预测的 I/O 带宽波动问题。

Acknowledgments

本工作部分得到美国能源部（DOE）高级科学计算研究办公室（ASCR）合同 DEAC02–06CH11357/0F–60169 和美国国家科学基金会（NSF）资助编号 2411386/2411387、2106635 的支持。本文中展示的结果是利用阿贡国家实验室的高性能计算系统——ALCF Polaris、联合系统评估实验室（JLSE），以及 NSF 的 CloudLab 和 Chameleon 实验床获得的。

References

- [1] Amazon Web Services. 2025. Amazon FSx for Lustre Pricing. <https://aws.amazon.com/fsx/lustre/pricing/>. Accessed: 2025-02-05.
- [2] Argonne Joint Laboratory for System Evaluation (JLSE). 2023. NVIDIA H100. <https://www.jlse.anl.gov/nvidia-h100>. Accessed: 2025-02-04.
- [3] Argonne Leadership Computing Facility. 2025. Polaris. <https://www.alcf.anl.gov/polaris>.
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] Zhenkun Cai, Xiao Yan, Kaihao Ma, Yidi Wu, Yuzhen Huang, James Cheng, Teng Su, and Fan Yu. 2021. Tensoropt: Exploring the tradeoffs in distributed dnn training with auto-parallelism. *IEEE Transactions on Parallel and Distributed Systems* 33, 8 (2021), 1967–1981.
- [6] Xiaoyu Chu, Daniel Hofstätter, Shashikant Ilager, Sacheendra Talluri, Duncan Kampert, Damian Podareanu, Dmitry Duplyakin, Ivona Brandic, and Alexandru Iosup. 2024. Generic and ML Workloads in an HPC Datacenter: Node Energy, Job Failures, and Node-Job Analysis. In *International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 710–719.
- [7] DeepSpeed Team. n.d.. Memory Requirements- DeepSpeed Documentation. <https://deepspeed.readthedocs.io/en/latest/memory.html>. Accessed: 2025-04-09.
- [8] Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. 2022. Understanding modern storage APIs: a systematic study of libaio, SPDK, and io_uring. In *Intl. Conference on Systems and Storage (SYSTOR)*. ACM, 120–127.
- [9] Jiangfei Duan, Shuo Zhang, Zerui Wang, Lijuan Jiang, Wenwen Qu, Qinghao Hu, Guoteng Wang, Qizhen Weng, Hang Yan, Xingcheng Zhang, Xipeng Qiu, Dahua Lin, Yonggang Wen, Xin Jin, Tianwei Zhang, and Peng Sun. 2024. Efficient Training of Large Language Models on Distributed Infrastructures: A Survey. [arXiv:2407.20018 \[cs.DC\]](https://arxiv.org/abs/2407.20018) <https://arxiv.org/abs/2407.20018>
- [10] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. 2024. AI and memory wall. *Micro* 44, 3 (2024), 33–39.
- [11] Mikaila J. Gossman, Bogdan Nicolae, and Jon C. Calhoun. 2024. Scalable I/O aggregation for asynchronous multi-level checkpointing. *Future Generation Computer Systems* 160 (2024), 420–432.
- [12] Matt Hamblen. 2023. Update: ChatGPT runs 10K Nvidia training GPUs with potential for thousands more. <https://www.fiercetechnics.com/sensors/chatgpt-runs-10k-nvidia-training-gpus-potential-thousands-more>.
- [13] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, Hyoungho Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. *GPipe: efficient training of giant neural networks using pipeline parallelism*. Curran Associates Inc., Red Hook, NY, USA.
- [14] Hongsun Jang, Jaeyong Song, Jaewon Jung, Jaeyoung Park, Youngsok Kim, and Jinho Lee. 2024. Smart-Infinity: Fast Large Language Model Training using Near-Storage Processing on a Real System. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 345–360.
- [15] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2024. NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference. [arXiv:2411.01142 \[cs.DC\]](https://arxiv.org/abs/2411.01142) <https://arxiv.org/abs/2411.01142>
- [16] Yao Jiang, Xinyu Yan, Ge-Peng Ji, Keren Fu, Meijun Sun, Huan Xiong, Deng-Ping Fan, and Fahad Shahbaz Khan. 2024. Effectiveness assessment of recent large vision-language models. *Visual Intelligence* 2, 1 (2024), 17.
- [17] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [18] Joonhyung Lee, Jeongin Bae, Byeongwook Kim, Se Jung Kwon, and Dongsoo Lee. 2024. To FP8 and Back Again: Quantifying the Effects of Reducing Precision on LLM Training Stability. [arXiv:2405.18710 \[cs.LG\]](https://arxiv.org/abs/2405.18710) <https://arxiv.org/abs/2405.18710>
- [19] Shenggui Li, Hongxin Liu, Zhengda Bian, Jiarui Fang, Haichen Huang, Yuliang Liu, Boxiang Wang, and Yang You. 2023. Colossal-ai: A unified deep learning system for large-scale parallel training. In *Proceedings of the 52nd International Conference on Parallel Processing*. 766–775.
- [20] Xiang Li, Yiqun Yao, Xin Jiang, Xuezhi Fang, Chao Wang, Xinzhang Liu, Zihan Wang, Yu Zhao, Xin Wang, Yuyao Huang, Shuangyong Song, Yongxiang Li, Zheng Zhang, Bo Zhao, Aixun Sun, Yequan Wang, Zhongjiang He, Zhongyuan Wang, Xuelong Li, and Tiejun Huang. 2024. Tele-FLM Technical Report. [arXiv:2404.16645](https://arxiv.org/abs/2404.16645)
- [21] Changyue Liao, Mo Sun, Zihan Yang, Kaiqi Chen, Binhang Yuan, Fei Wu, and Zeke Wang. 2024. Adding NVMe SSDs to Enable and Accelerate 100B Model Fine-tuning on a Single GPU. *arXiv preprint arXiv:2403.06504* (2024).
- [22] Changyue Liao, Mo Sun, Zihan Yang, Jun Xie, Kaiqi Chen, Binhang Yuan, Fei Wu, and Zeke Wang. 2024. LoHan: Low-Cost High-Performance Framework to Fine-Tune 100B Model on a Consumer GPU. [arXiv:2403.06504 \[cs.DC\]](https://arxiv.org/abs/2403.06504) <https://arxiv.org/abs/2403.06504>
- [23] Aixun Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bocho Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [24] Avinash Maurya, Robert Underwood, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. DataStates-LLM: Lazy Asynchronous Checkpointing for Large Language Models. In *Proc. of the International Symposium on High-Performance Parallel and Distributed Computing (Pisa, Italy) (HPDC'24)*. 13 pages.
- [25] Avinash Maurya, Jue Ye, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. Breaking the Memory Wall: A Study of I/O Patterns and GPU Memory Utilization for Hybrid CPU-GPU Offloaded Optimizers. In *FlexScience'24: Workshop on AI & Scientific Computing at Scale using Flexible Comp. Infrastructures*.
- [26] Avinash Maurya, Jie Ye, M Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. Deep optimizer states: Towards scalable training of transformer models using interleaved offloading. In *Proceedings of the 25th International Middleware Conference*. 404–416.
- [27] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. [arXiv:1710.03740 \[cs.AI\]](https://arxiv.org/abs/1710.03740) <https://arxiv.org/abs/1710.03740>

- [28] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, et al. 2022. Scaling Language Models: Methods, Analysis & Insights from Training Gopher. *arXiv:2112.11446 [cs.CL]* <https://arxiv.org/abs/2112.11446>
- [29] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory optimizations Toward Training Trillion Parameter Models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–16.
- [30] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: breaking the GPU memory wall for extreme scale deep learning. In *SC'21: The 2021 International Conference for High Performance Computing, Networking, Storage and Analysis*. St. Louis, USA, Article 59, 14 pages.
- [31] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3505–3506.
- [32] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 551–564.
- [33] Mohammad Shoyebi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [34] Shuaiwen Leon Song, Bonnie Krufft, Minjia Zhang, Conglong Li, Shiyang Chen, et al. 2023. DeepSpeed4Science Initiative: Enabling Large-Scale Scientific Discovery through Sophisticated AI System Technologies. *arXiv:2310.04610 [cs]*
- [35] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A Large Language Model for Science. *arXiv:2211.09085 [cs.CL]* <https://arxiv.org/abs/2211.09085>
- [36] DeepSpeed Team. 2022. GitHub Issue: Delayed parameter update in ZeRO-Offload. <https://github.com/deepspeedai/DeepSpeed/discussions/2461>.
- [37] HPCAI Tech. 2025. TensorNVMe: A Python library for transferring PyTorch tensors between CPU and NVMe. <https://github.com/hpcaitech/TensorNVMe>. Accessed: 2025-02-06.
- [38] Alan Thompson. [n.d.]. Inside Large Language Models. <https://lifearchitected.ai/models>. Accessed: 2025-04-12.
- [39] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288 [cs]*
- [40] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288*
- [41] Jonathan Vanian and Kif Leswing. [n.d.]. ChatGPT and generative AI are booming, but the costs can be extraordinary. <https://www.cnbc.com/2023/03/13/chatgpt-and-generative-ai-are-booming-but-at-a-very-expensive-price.html>. Accessed: 2023-03-15.
- [42] VAST Data. 2023. The VAST Data Platform Whitepaper. <https://www.vastdata.com/whitepaper>. Accessed: 2025-04-12.
- [43] Guanhua Wang, Masahiro Tanaka, Xiaoxia Wu, Lok Chand Koppaka, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2023. DeepSpeed ZeRO-Offload++: 6x Higher Training Throughput via Collaborative CPU/GPU Twin-Flow.
- [44] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, TS Eugene Ng, and Yida Wang. 2023. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 364–381.
- [45] BigScience Workshop. 2023. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv:2211.05100 [cs]*
- [46] Gang Xian, Wenxiang Yang, Yusong Tan, Jinghua Feng, Yuqi Li, Jian Zhang, and Jie Yu. 2024. Mobilizing underutilized storage nodes via job path: A job-aware file striping approach. *Parallel Comput.* 121 (2024), 103095.
- [47] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, et al. 2022. GLM-130b: An open bilingual pre-trained model. *arXiv preprint arXiv:2210.02414* (2022).
- [48] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, Weng Lam Tam, Zixuan Ma, Yufei Xue, Jidong Zhai, Wenguang Chen, Peng Zhang, Yuxiao Dong, and Jie Tang. 2023. GLM-130B: An Open Bilingual Pre-trained Model. *arXiv:2210.02414 [cs.CL]* <https://arxiv.org/abs/2210.02414>
- [49] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
- [50] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM computing surveys (CSUR)* (2019).
- [51] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
- [52] Xia Zhao, Limin Wang, Yufei Zhang, Xuming Han, Muhammet Deveci, and Milan Parmar. 2024. A review of convolutional neural networks in computer vision. *Artificial Intelligence Review* 57, 4 (2024), 99.
- [53] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3848–3860.